A Constraint-Aware Production Planner

Deciding what a factory can actually make when demand outruns shared supply — and exactly what to do about each shortfall.

Portfolio case study · prepared by **Jay Clouse** · 2026 | Identifiers, data and the optimization method are redacted or illustrative; this is a public, non-proprietary overview.

I designed and built a decision-support tool that answers a question manufacturers face every day and spreadsheets answer badly: **when there isn't enough of the shared ingredients to make everything that was ordered, what *can* we make, what is blocking each shortfall, and what single action fixes the most?**

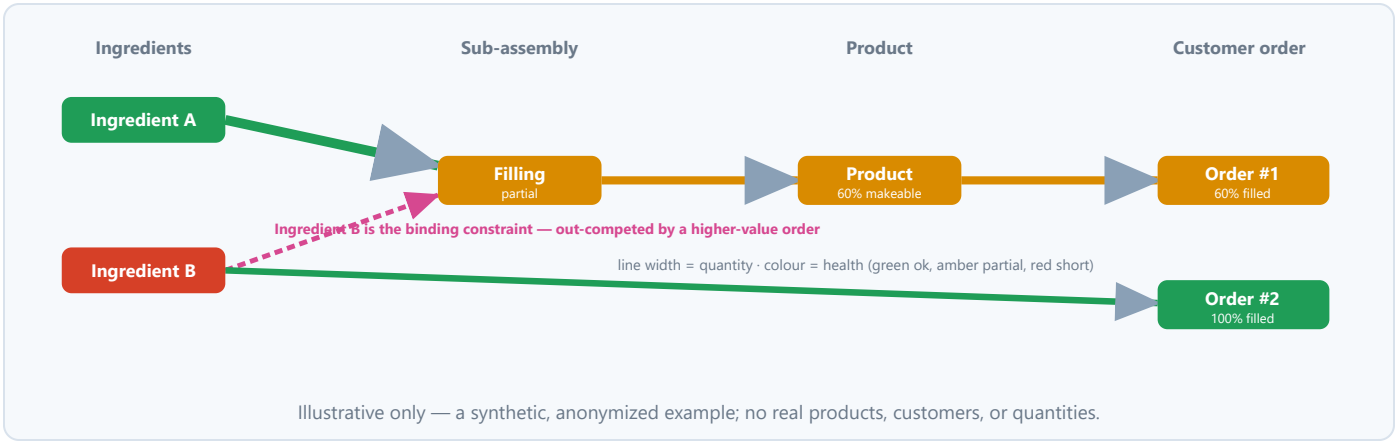
The problem

A food manufacturer runs hundreds of products that draw on the same pool of limited ingredients, made across several plants, with inventory that expires and purchase orders that arrive over time. In a shortage, a planner has to decide — quickly, by hand — which customer orders get the scarce material, which fall short, and what to do: buy more, move stock between sites, re-prioritize, or schedule a missing step. Standard material-planning (MRP) explodes demand as if the plant could make anything, and treats each product independently — so it never answers "what can we actually build right now, and *why* not?"

What I built

A planner that reads the current order book (customer orders, inventory, recipes, purchase orders, work orders) and computes, across a **90-day horizon**, a realistic, conserved production plan — then explains it. For every work order it reports how much is genuinely makeable; for every shortfall it names the real constraints, in priority order, each paired with the highest-leverage fix; and it puts a dollar figure on what the shortage is costing. A color-coded, interactive map lets a planner click a customer order and trace it down to the exact raw material holding it back.

Capability	What it delivers
True feasibility	How much of each order can actually ship, time-phased over 90 days — honoring recipes, batch sizes, plant boundaries, lot expiration, and when supply really arrives.
Root-cause, in layers	The constraints behind each shortfall are rarely one thing. The tool lists them in priority order — fix the top one and the next is revealed — each with a specific lever: buy or expedite, transfer stock between sites, re-prioritize, schedule a missing step, or use fresher stock.
Value-based allocation	Steers scarce ingredients to the most valuable demand, quantifies revenue at risk, and ranks the shortages so the one dollar spent unlocks the most sales.
Stranded inventory	Flags stock sitting at a site a given order can't draw from — turning a "buy more" into a cheaper internal transfer.
What-if, re-solved	Test a change — cut an order, expedite a shipment, move stock, reschedule a run — and see the full downstream ripple before committing, computed by a real re-solve rather than a linear guess.
Interactive map	A browser view of the whole supply network, colored by health, drillable from a customer order to its binding ingredient, with the financial impact updating live.



Illustrative only — a synthetic, anonymized example; no real products, customers, or quantities.

Why it beats standard planning

Classic MRP

Explodes demand assuming the plant can make anything; treats each item on its own; no notion of which shortage matters most.

This planner

Resolves competition for shared scarce ingredients; tells you what is actually makeable and why not; ranks the fixes by the revenue each unlocks.

In short, it converts a long, undifferentiated shortage list into a **prioritized, dollar-ranked action plan** — and lets a planner test the action before committing it in the system of record.

Status & impact — an honest note

The system is **complete and working**: given the real order book and master data it produces full, conserved 90-day plans with the diagnostics and what-if tools described above, validated end-to-end on a realistic test dataset. It was **never adopted as the live production planner**, so its real-world savings were **never formally measured** — I make no claim to a specific dollar or time figure, because none was ever captured.

The reason is organizational, not technical: running it for real would mean **switching the plant off its established MRP process and re-pegging live production onto a new, unproven, solo-built tool** — a step that requires executive authority and an appetite for risk that such a tool is rarely given, and proving the savings would itself require that leap first. That is a common fate for ambitious internal tools; I think it is more useful to be straight about it than to invent a number.

What the tool *does* do is concrete and demonstrable:

- **Names the cause of every shortfall** — the binding constraints in priority order, each with a specific lever — where a spreadsheet leaves a planner guessing.
- **Points to the highest-return action** — the expedite or transfer that unlocks the most sales — rather than whichever shortage is loudest.
- **Separates "missing" from "stranded"** — stock sitting at the wrong site is flagged as a transfer, not a purchase.
- **Lets a decision be tested before it is taken** — every what-if is a real re-solve, so the ripple is visible up front.

How I would prove it — a proposed trial

Since the savings were never measured, here is the experiment I would run to measure them honestly — designed to be low-risk, seasonally fair, and hard to fool:

- **Limited, reversible scope.** Run the tool for a single **division** (or a defined sample of products) over one 90-day period — not the whole plant — so the trial is contained and can be rolled back.
- **Season-matched baselines.** Build the comparison from that division's historical 90-day periods, including the **same seasonal window across several prior years**, so the trial is judged against like-for-like demand rather than an arbitrary quarter.
- **A metric chosen in advance.** Fix the success measure up front — **order fill rate**, **value fill** (fill weighted by revenue), or **profit margin** — and test the trial period against the seasonal baseline with a **t-test** for significance.
- **A control group, before and after.** Compare the **tool-run sample** against the **rest of the plant** (still on the existing process), both *before* and *during* the trial — a difference-in-differences read. If the sample's fill improves while the non-sample's does not, the gain is the tool's, not a plant-wide good season.

> The design deliberately separates the tool's effect from seasonality and from plant-wide swings, and keeps the commitment small and reversible — the kind of contained, measured trial that could earn the trust a full switch-over would require.

How it is built

The tool has four parts: a **data layer** that integrates with the company's ERP (JD Edwards) to read the live order book and master data; a custom **optimization engine** that turns it into a time-phased, conserved production plan; a lightweight **service** that answers questions about the plan; and an **interactive visualization** in the browser. It is packaged as a single application a planner can run without any technical setup.

> The **optimization method itself is the proprietary core** of this work and is intentionally not described here. What is shown above is the *problem it solves*, the *decisions it supports*, and the *value it delivers* — not the recipe.

How I work — and what this demonstrates

If I claim expertise at anything, it is **systems thinking, troubleshooting, and solving problems by what actually works — not by what the instruction book claims.** This project is a direct instance of that: standard planning follows the textbook and assumes the plant can make anything; I rebuilt it around what the plant actually experiences under real, shared-supply constraints. The specific skills below are the evidence, not the identity.

Systems thinking

Modeling a whole plant as one connected, time-phased flow — recipes, inventory, sites, timing and competing demand — and reasoning about how a change in one place ripples through everything else.

Troubleshooting & empirical rigor

Diagnosing *why* the standard approach fails in practice, then building and testing what holds up against reality — keeping what works and discarding what only works on paper.

Applied optimization

Framing and solving a large constrained allocation problem with competing demands — learning exactly the mathematics a problem requires and making it work in a live system.

Full-stack, end-to-end delivery

Sole ownership of the whole stack: data ingestion from an enterprise ERP, the engine, a service layer, an interactive UI, and a packaged one-click app for non-technical users.

This document is a public overview of a personal project. All product, customer, plant and supplier identifiers have been removed; any figures and the supply map are synthetic and illustrative; and the optimization method is omitted to protect intellectual property. It is safe to share and contains nothing proprietary.